

RTOS Task Scheduling

PROGRAM

```
#include <stdio.h>
```

```
typedef struct {  
    char name[20];  
    int priority;  
    int execution_time;  
} Task;
```

```
int main() {  
    Task tasks[] = {  
        {"Task1", 3, 5},  
        {"Task2", 1, 3},  
        {"Task3", 2, 4}  
    };  
};
```

```
int n = 3;
```

```
printf("RTOS Task Scheduling in C\n");
```

```
printf("Execution Order:\n");
```

```
// Higher priority executes first
```

```
for (int p = 3; p >= 1; p--) {  
    for (int i = 0; i < n; i++) {  
        if (tasks[i].priority == p) {  
            printf("%s -> Priority: %d, Time: %d ms\n",
```

```

        tasks[i].name,
        tasks[i].priority,
        tasks[i].execution_time);
    }
}
}

return 0;
}

```

OUTPUT

The screenshot shows the Programiz Online C Compiler interface. The code in the editor is as follows:

```

9 int main() {
10     int n = 3,
17
18     printf("RTOS Task Scheduling in C\n");
19     printf("Execution Order:\n");
20
21     // Higher priority executes first
22     for (int p = 3; p >= 1; p--) {
23         for (int i = 0; i < n; i++) {
24             if (tasks[i].priority == p) {
25                 printf("%s -> Priority: %d, Time: %d ms\n",
26                     tasks[i].name,
27                     tasks[i].priority,
28                     tasks[i].execution_time);
29             }
30         }
31     }
32
33     return 0;
34 }

```

The output window displays the following results:

```

RTOS Task Scheduling in C
Execution Order:
Task1 -> Priority: 3, Time: 5 ms
Task3 -> Priority: 2, Time: 4 ms
Task2 -> Priority: 1, Time: 3 ms

```

Below the output, it states "=== Code Execution Successful ===".

PROGRAM

```

#include <iostream>

#include <vector>

#include <algorithm>

```

```

using namespace std;

struct Task {
    string name;
    int priority;
    int execution_time;
};

int main() {
    vector<Task> tasks = {
        {"Task1", 3, 5},
        {"Task2", 1, 3},
        {"Task3", 2, 4}
    };

    sort(tasks.begin(), tasks.end(),
        [](Task a, Task b) {
            return a.priority > b.priority;
        });

    cout << "RTOS Task Scheduling in C++\n";
    cout << "Execution Order:\n";

    for (auto t : tasks) {
        cout << t.name
            << " -> Priority: " << t.priority
            << ", Time: " << t.execution_time << " ms\n";
    }
}

```

```

return 0;

}

```

OUTPUT

The screenshot shows the EDA Playground web interface. The left sidebar contains navigation links for Languages & Libraries, Tools & Simulators, and Examples. The main editor area displays two files: `testbench.cpp` and `design.cpp`. The `testbench.cpp` file contains C++ code for creating and scheduling three tasks (Task1, Task2, Task3) with different priorities and execution times. The `design.cpp` file is empty. The bottom panel shows the execution log, which includes the command used to compile and run the code, and the output of the program, showing the execution order and timing for each task.

```

// testbench.cpp
15  {"Task2", 1, 3},
16  };
17
18  sort(tasks.begin(), tasks.end(),
19  [](Task a, Task b) {
20      return a.priority > b.priority;
21  });
22
23  cout << "RTOS Task Scheduling in C++\n";
24  cout << "Execution Order:\n";
25
26  for (auto t : tasks) {
27      cout << t.name
28          << " -> Priority: " << t.priority
29          << ", Time: " << t.execution_time << " ms\n";
30  }
31
32  return 0;
33

```

```

// design.cpp
1  // Code your design here.
2  // Uncomment the next line for SystemC modules.
3  // #include <systemc>

```

```

[2026-08-25 16:56:37 UTC] g++ -o sim *.cpp && echo "Compile done. Starting run..." && ./sim
Compile done. Starting run...
RTOS Task Scheduling in C++
Execution Order:
Task1 -> Priority: 3, Time: 5 ms
Task3 -> Priority: 2, Time: 4 ms
Task2 -> Priority: 1, Time: 3 ms
Done

```

RESULT

C provides a simple and lightweight implementation suitable for resource-constrained RTOS systems, while C++ provides easier task management through structures, vectors, sorting, and higher-level programming features.